

开发指南

对应任务书：九.6 程序计算流程。
面向后续维护与二次开发，说明开发环境、脚本、计算流程与代码约定。

一、开发环境

工具	版本 / 说明
Python	3.12（建议 conda 环境 concrete）
Node.js	20+（前端构建用）
包管理	pip（后端） / npm（前端）

```
# 后端环境
conda create -n concrete python=3.12 -y && conda activate concrete
cd backend && pip install -r requirements-dev.txt

# 前端环境
cd frontend && npm install
```

requirements-dev.txt 含 -r requirements.txt，并附加 pytest、httpx。
pyproject.toml 仅保留 [tool.pytest.ini_options]。

二、常用脚本

后端

命令	说明
uvicorn app.main:app --reload --port 8000	开发服务器（热重载）
pytest -v	全部测试
pytest tests/test_teacher_validation.py -v	仅教师回归

<code>pytest -k beam -v</code>	按关键字筛选
--------------------------------	--------

前端

命令	说明
<code>npm run dev</code>	Vite 开发服务器（HMR，:3000，代理 /api → :8000）
<code>npm run build</code>	生产构建（ <code>vue-tsc</code> 类型检查 + <code>vite build</code> ）
<code>npm run preview</code>	本地预览 dist
<code>npm run test</code>	Vitest 监听模式
<code>npm run test:run</code>	Vitest 单次运行（CI）

打包 / 部署

命令	说明
<code>python build_win.py</code>	构建 Windows 便携版（需 <code>pip install pyinstaller</code> ）
<code>docker compose up -d --build</code>	Docker 部署（详见 安装与运行 ）

三、计算流程（后端）

POST /api/projects/{id}/calculate 的处理流程
([backend/app/api/projects.py](#)):

1. 按 `user_id + project_id` 取项目 data
2. 校验 `STRUCTURE_REQUIRED` (13 项) + `LOADS_REQUIRED` (5 项) 是否齐全
3. 派生构件输入 (`solvers/derive.py`):
板 $\leftarrow$ `derive_slab_input(structure, loads)`
次梁 $\leftarrow$ `derive_beam_input(structure, loads)`
主梁 $\leftarrow$ `derive_main_beam_input(structure, loads)`
4. 调用编排函数 (材料常量由 `materials.py` 注入):
`calculate_slab(...)` $\rightarrow$ 荷载 $\rightarrow$ 跨度 $\rightarrow$ 内力 $\rightarrow$ 正截面
`calculate_beam(...)` $\rightarrow$ 荷载 $\rightarrow$ 跨度 $\rightarrow$ 内力 (边缘调整) $\rightarrow$ 正截面 (T/矩) $\rightarrow$ 斜截面
`calculate_main_beam(...)` $\rightarrow$ 集中力 $\rightarrow$ 包络 (边缘调整) $\rightarrow$ 正截面 (T) $\rightarrow$ 斜截面+吊筋
5. 结果写回 `data.slab/beam/main_beam.result`, 置 `initialized=true`
6. 返回类型化结果; 超筋 (`ValueError`) $\rightarrow$ 400

派生关系 (荷载自动传递):

- 次梁间距 = 板单跨 = $L2 / \text{板跨数}$;
- 次梁跨度 = $L1 / \text{次梁跨数}$;
- 主梁跨度 = $L2 / \text{主梁跨数}$;
- 板 / 次梁 > 5 跨 $\rightarrow$ 五跨连续简化 (derive 传 `min(spans, 5) + width = \text{单跨} \times \text{模型跨数}`)。

四、前端计算流程

- 参数页确认 $\rightarrow$ `useProject.calculate(page)`: 先 `saveNow()` 刷新 $\rightarrow$ `POST /calculate` $\rightarrow$ 合并结果 $\rightarrow$ 自动保存。
- 级联: 前端顺序 `calculate('slab') \rightarrow ('beam') \rightarrow ('main_beam')` (三构件独立可算, 无运行时数据依赖)。
- 计算书: `buildReportDoc(data)` 纯函数把只读 `data` (含手改) 装配为章节树, `ReportDocument.vue` 渲染。

五、代码约定

- 后端: 分层 (`api / auth / solvers / data`), 求解器纯函数化、每阶段 Pydantic 模型; 新增需隔离的端点必须 `Depends(get_current_user)` 并按 `user_id` 存取。
- 前端: Composition API + 模块级单例 (不引 Pinia); 只用语义令牌 (`var(--primary)` 等), 不硬编码颜色; 新增页面基于 `PageHeader` 扩展; 前端任务必跑 `npm run build`。
- 材料常量: 后端 `materials.py` 与前端 `report/materials.ts` 双侧同步, 改动任一侧需同步另一侧。

- **测试**：新功能附单测；求解器改动跑 `test_teacher_validation.py` 防回归。

六、关键文件速查

关注点	文件
计算公式	<code>backend/app/solvers/common.py</code>
荷载派生	<code>backend/app/solvers/derive.py</code>
板 / 次梁 / 主梁编排	<code>solvers/{slab,beam,main_beam}/solver.py</code>
材料常量	<code>backend/app/materials.py</code>
计算 API	<code>backend/app/api/projects.py</code>
计算书装配	<code>frontend/src/composables/useReportDocument.ts</code>
三主题令牌	<code>frontend/src/assets/styles/tokens.css</code>
数据形状	<code>frontend/src/api/projects.ts</code>
教师回归	<code>backend/tests/test_teacher_validation.py</code>